DEVELOPER OFFICE HOURS

AI Agents in Production

From architecture to a production-grade deployment

Ruth Mutua · Prescott Data

Developer Office Hours · 22 September 2026

What you'll leave with

1

A reference architecture

Four planes, and the peer mesh that ties the agents together.

2

What agents ask of a platform

The requirements, and how they decided where we run.

3

A setup order that works

The sequence, and the handful of settings that matter.

4

A readiness checklist

Ten questions to ask before agents go live.

— Everything I show comes from running JarvisCore, our open-source agent framework, in production. Product and tenant names are generalized.

Why agents aren't just another service

TRAIT	WHAT IT MEANS FOR OPERATIONS
Long-running	No request timeouts in front; graceful drain on deploy
Stateful	Stable identity and durable state, or a restart loses work
Non-deterministic	You can't assert outputs, so you bound behaviour instead
Holds credentials to other systems	Blast radius is every system it can reach
Multi-tenant	Isolation has to be designed in, not added later
Expensive per call	Budgets, quotas and fallbacks are reliability features

Agenda

01	**Anatomy**	The four planes, the peer mesh, and the controls around every agent
02	**Platform**	What the workload asks for, the options, and where we run
03	**Setup**	The order that works, and the configuration that matters
04	**Running it**	Watching it, keeping it up, and what went wrong
05	**Live demo**	One request through a production control room
06	**Close**	A checklist to take back to your own platform

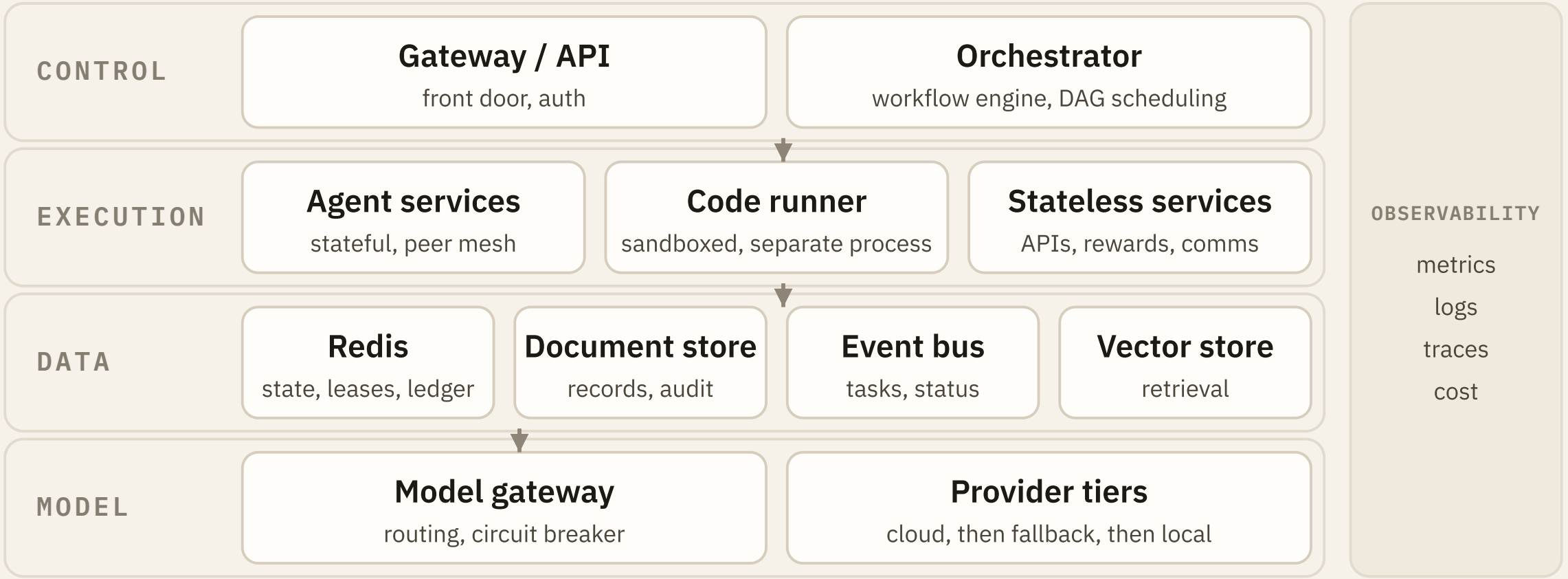
01

Anatomy

The four planes, the peer mesh, and the controls around every agent

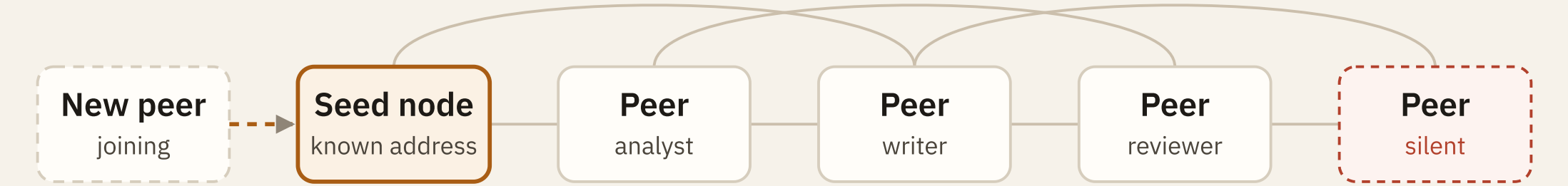
6 Reference architecture: four planes
7 Agents as peers: mesh, seeds and gossip
8 Why gossip, and what it costs
9 Security: layers, not a single gate
10 Memory: four tiers, one interface
11 Tool access: one door, built in
12 Orchestration: leases and fencing

Reference architecture: four planes



— What we run: two stateful agent services on a gossip mesh, a stateless service, a model gateway, and a managed data layer.

Agents as peers: mesh, seeds and gossip



Peer
A running agent service that advertises what it can do.

Mesh
All the peers that know about each other. No central list.

Seed node
A known address a new peer contacts to join. An entry point, not a leader.

Gossip (SWIM)
Peers ping each other and share news, so everyone converges on who's alive.

— Membership, failure detection and capability announcements ride on gossip. Agent-to-agent messages go peer to peer.

Why gossip, and what it costs

What you get

- No registry or broker to run, scale or fail
- Failure detection built in: silent peers go suspect, then dead
- Capabilities announced: the roster writes itself, no redeploy
- No permanent coordinator: any eligible peer can claim work
- The same code from one process to a fleet

What it asks of your platform

- Stable identity per node: a seed needs a stable name
- TCP and UDP open on the mesh ports
- Membership is eventually consistent, not instant
- Work and state still live in Redis: gossip is not the source of truth

— If you've run Consul or Cassandra, you've relied on the same idea. The right-hand column is why the platform choice comes next.

Security: layers, not a single gate

LAYER	CONTROL
Identity	Workload identity to a managed secrets store
Credentials	Nexus, a broker at the call boundary; agents hold opaque handles, never tokens
Code execution	Generated code runs in a separate process with a scrubbed environment
Network	Policies scoped per service, opened per port and protocol
Tenants	Tenant id carried in every memory and workflow key
Actions	Irreversible tools require human approval

— Direction we're hardening toward: authenticate at the endpoint, not only at the network boundary.

Memory: four tiers, one interface

Working

Scratchpad for the current step, discarded when it finishes.

`Lives in: blob storage`
`Lifetime: the step`

Episodic

An append-only ledger of what happened, in order. The audit trail.

`Lives in: a Redis stream`
`Lifetime: 7 days by default`

Long-term

The model compresses a long ledger into a summary, backed up to blob storage.

`Lives in: Redis + blob`
`Lifetime: the workflow`

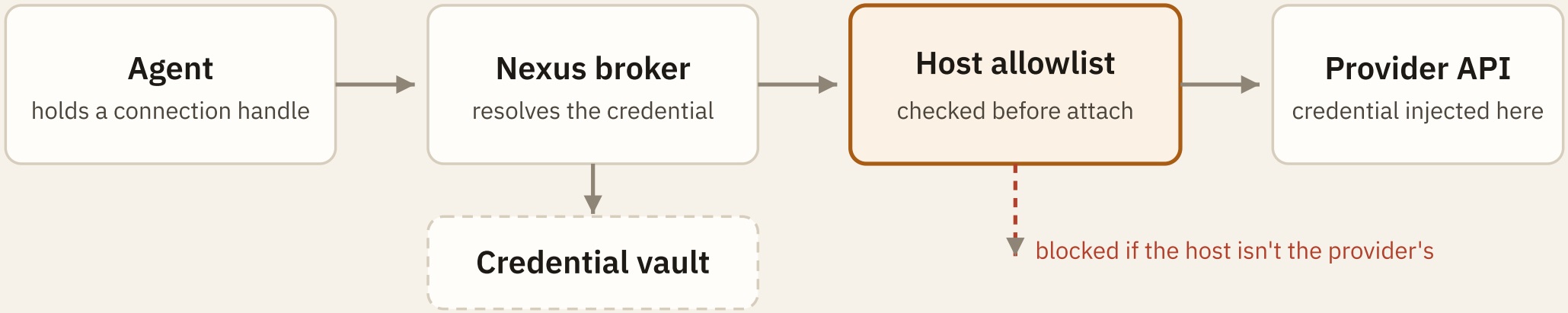
Cross-session

Semantically searchable memory that spans workflows and restarts, not just the current one. A restarted agent recovers what it learned, not just its last checkpoint.

`Lives in: an external memory service`
`Lifetime: across sessions`

— One object, UnifiedMemory, fronts all four tiers. For this tier, we use Athena, our open-source memory framework — it ships with three tiers of its own (short-term, mid-term, long-term graph). Set ATHENA_URL and every agent upgrades, with no code changes.

Tool access: one door, built in



Nexus: one call for OAuth2, API keys and Basic auth, with automatic token refresh

About 150 integrations ship as atoms: typed single-file functions the agent finds in a registry

Destructive tools declare an approval policy; execution is at-least-once, so keep tools idempotent

— In JarvisCore, agents never hold a token. Nexus checks the host allowlist before a credential is attached.

Orchestration: leases and fencing



A workflow is a DAG in Redis; workers claim a step with a leased lock

A heartbeat renews the lease; an expired lease is recovered and re-queued

Only the current claim holder can commit a result

— Think of a job queue with a visibility timeout, plus a fencing token. No permanent coordinator: any eligible peer can claim a ready step.

02

Platform

What the workload asks for, the options, and where we run

14 What the workload asks of the platform

15 The options and the decision

16 The resulting topology

What the workload asks of the platform

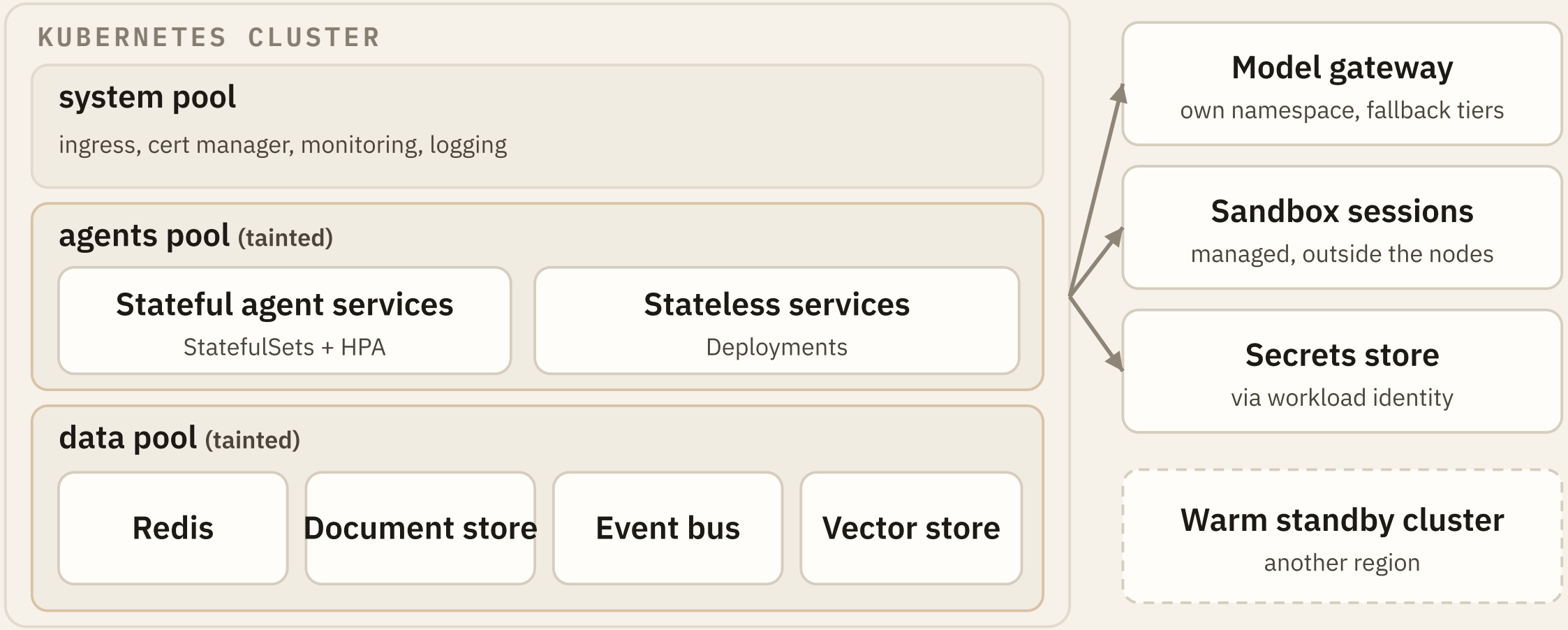
WORKLOAD TRAIT	PLATFORM REQUIREMENT
Stateful gossip mesh	Stable pod identity and per-pod DNS
Long-running steps	Graceful termination; no idle timeout in front
Bursty load	Autoscaling that scales down slowly
Slow startup	Generous rollout timeouts and ordered start
Untrusted generated code	Isolation outside the cluster nodes
Multi-tenant	Namespace and network boundaries
Model capacity limits	A gateway with fallback tiers

The options and the decision

OPTION	WHERE IT FITS	WHERE IT STRUGGLES
Serverless functions	Stateless, short, bursty work	Long-running, stateful, identity-bound agents
Container-app platforms	Simple long-running containers	Less control over pod identity, placement and policy
Kubernetes	StatefulSets, node pools, network policy, tuned autoscaling	You operate it
VMs + compose	Where we started: fast to stand up	Rolling updates, autoscaling, isolation at scale

DECISION We chose Kubernetes: StatefulSets for the mesh, node pools for isolation, and a managed sandbox-session service for untrusted code. It also sat next to the platform we already ran.

The resulting topology



— Separate clusters per product shape, with a warm standby in another region.

03

Setup

The order that works, and the configuration that matters

18 Setup order, and why the order matters

19 The configuration that matters

Setup order, and why the order matters

1. **Infrastructure as code**
   Cluster, workload identity, secrets store, registry
2. **Platform layer**
   Cert manager, ingress, monitoring, logging
3. **Data layer**
   Redis, document store, event bus, vector store
4. **Agent services**
   Seed pod first, then the rest in order
5. **Policies and autoscaling**
   Network policies and HPA, once the workload is healthy
6. **CI/CD**
   Build, then roll out with a long timeout: agent startup is slow

— Agents assume the data layer and their seed peer already exist.

The configuration that matters

STABLE IDENTITY

```
kind: StatefulSet
spec:
  serviceName: agents-headless
  podManagementPolicy: OrderedReady
  # ordinal 0 is the seed
  containers:
  - env:
    - name: SWIM_BIND_HOST
      valueFrom: {fieldRef:
        {fieldPath: status.podIP}}
      # not 0.0.0.0
```

Peers find each other by name, and the seed pod starts first.

SLOW TO SHRINK

```
kind: HorizontalPodAutoscaler
spec:
  minReplicas: 3
  maxReplicas: 10
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
      policies: [{type: Pods,
        value: 1, periodSeconds: 120}]
  # VPA: InPlaceOrRecreate
```

Every pod that leaves costs the mesh time to reconverge.

BOTH PROTOCOLS

```
kind: NetworkPolicy
  # mesh ports, ingress + egress
  - ports:
    - {protocol: UDP, port: 8001}
      # steady-state gossip
    - {protocol: TCP, port: 8001}
      # joins and full sync
```

Joins use TCP; steady-state gossip uses UDP. Allow both.

04

Running it

Watching it, keeping it up, and what went wrong

21 Observability and cost control

22 Staying up: resilience patterns

23 Challenges and what we learned

Observability and cost control

What we watch

- Workflow and step state, from the ledger
- Lease expiries and reclaimed steps
- Pod memory near its limit; OOMKilled
- Mesh membership, not just readiness
- Tokens and cost per workflow
- Provider errors and rate limits

Who provides it

- JarvisCore: ledger stream, scrubbed traces, per-workflow budgets, optional metrics endpoint
- Operator: alert rules, dashboards, membership checks
- Stack: Prometheus and Grafana for metrics, Loki for logs

— JarvisCore gives you the events. You build the alerts.

Staying up: resilience patterns

PATTERN	WHAT IT PROTECTS AGAINST
Leases and fencing	A stalled or partitioned worker holding a step
Provider fallback, capped backoff	A model provider that is rate-limited or out of quota
Gateway circuit breaker and model tiers	Capacity exhausted in one tier
Human approval	An irreversible action taken on a bad plan
Idempotent tools	At-least-once execution repeating a side effect

Challenges and what we learned

CHALLENGE	WHAT IT TAUGHT US	WHAT WE DO NOW
Mesh nodes advertised an unroutable address	Bind to what peers can reach	Pod IP injected at start
A blocking call froze the API while gossip looked healthy	A healthy process isn't a working service	Blocking init runs off the event loop
Seed hostname cached through rolling restarts	Stable identity is not stable addresses	Check membership on every replica after a restart
Network policy allowed UDP but not TCP	Policies fail silently until a cold start	Test a full-restart join
Admission cap of 16 over a call limit of 1	Set concurrency limits explicitly, in one place	Explicit defaults in config

05

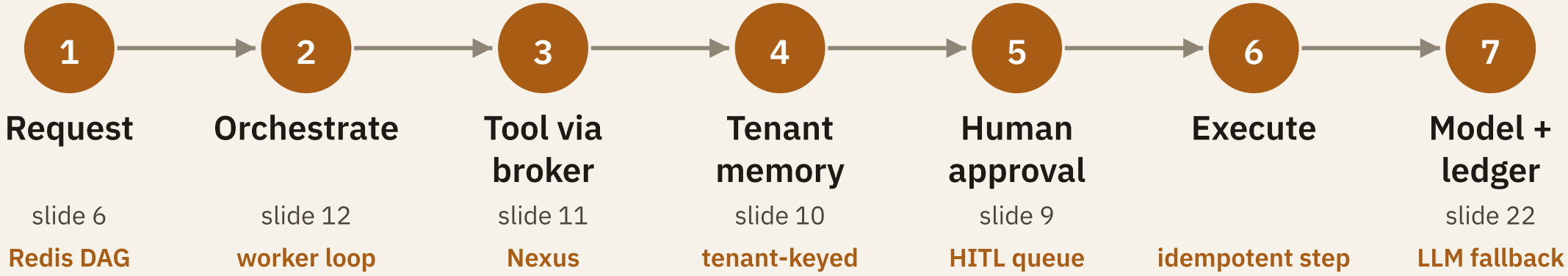
Live demo

One request through a production control room

25 One request, end to end

26 The production control room

One request, end to end



Real JarvisCore: Mesh workers, Nexus, HITL queue, LLM client with fallback

Stubbed: only the third-party API and the model providers

Then, simulated: a worker is lost, and the model provider hits capacity

— The path you're about to watch. Each stage maps to a slide you've just seen.

The production control room

WHAT TO WATCH FOR

1. Steps claimed by two workers, with lease timers
2. A tool call: the agent holds a handle, never a token
3. Two tenants with separate memory
4. An approval gate on an irreversible step
5. Simulate: worker lost
6. Simulate: model provider at capacity

```
Run it yourself: control_room/ in this folder. No LLM key needed.
```

06

Close

A checklist to take back to your own platform

28 The production-readiness checklist

29 Take it further

The production-readiness checklist

- [ ] **1. Identity**
  Workload identity; no shared secrets baked into images
- [ ] **2. Credentials**
  Brokered, never in agent context
- [ ] **3. Tenants**
  Isolation in every key and policy
- [ ] **4. Bounded behaviour**
  Iteration, time and token budgets
- [ ] **5. Idempotent tools**
  Safe to run twice
- [ ] **6. Leases and fencing**
  Work survives a lost worker
- [ ] **7. Budgets**
  Cost per workflow, alerts on drift
- [ ] **8. Fallbacks**
  Provider, model tier, degraded mode
- [ ] **9. Observability**
  State, membership and cost, not just readiness
- [ ] **10. Human gates**
  Approval for irreversible actions

Take it further

Run this session's code

`control_room/`
The live demo: two Mesh workers, Nexus, a human gate, failover

`beat1_lease_recovery.py`
A lost worker's step is reclaimed and finished

`beat2_nexus_contract.py`
The agent holds a handle; an off-allowlist host is refused

`beat3_failover.py`
A provider at capacity, and the fallback that takes over

The projects behind it

`pip install jarviscore-framework`
The agent framework used throughout

`github.com/Prescott-Data/jarviscore-framework`
Source, docs and examples

`github.com/Prescott-Data/athena`
Cross-session memory for agents